Experiment 6

Student Name: Ayush Tiwari **UID: 23BCS11366**
Branch: CSE **Section/Group: KRG 3-B**
Semester: 6th **Date of Performance:25/02/2026**
Subject Name: System Design **Subject Code: 23CSH-314**

Aim: To design and analyze a scalable video-sharing and streaming platform similar to YouTube that supports video uploads, adaptive streaming (HLS/DASH), likes, comments, and massive concurrent users with high availability and low latency.

Objective:

- To identify functional and non-functional requirements of a YouTube-like system
- To design High-Level Architecture (HLD) for a video-sharing platform
- To understand Manifest files and Adaptive Bitrate Streaming
- To implement Like and Comment handling in distributed systems
- To understand the role of CDN, Kafka, Cache, and Object Storage
- To apply scalability and CAP theorem concepts

Tools:

- **Draw.io** – System Architecture & HLD Diagram
- **MySQL** – User, subscription, and payment data
- **MongoDB / NoSQL DB** – Video metadata storage
- **ElasticSearch** – Search functionality for movies and TV shows
- **Apache Kafka** – Asynchronous event streaming and video pipeline coordination
- **Redis / CDN Cache** – Caching frequently accessed content • **Blob Storage (Amazon S3 equivalent)** – Video and media storage

System Requirements:

A. Functional Requirements:-

1. User can register and login
2. User can upload videos
3. System transcodes videos into multiple resolutions (360p, 720p, 1080p, 4K)
4. System generates HLS (.m3u8) or DASH (.mpd) manifest files 5. User can search videos
6. User can watch videos with adaptive streaming
7. User can like/unlike videos

8. User can comment and reply on videos
9. System displays real-time like count
10. System supports pagination of comments

B. Non-Functional Requirements:-

1. **Scalability**

- Target Users: 500M+
- Millions of concurrent video streams
- High write traffic for likes

2. **Availability vs Consistency (CAP Theorem)**

- Availability >>> Consistency **Reason**:
- Video playback must not stop
- Slight delay in like count update is acceptable
- Comments may be eventually consistent **Example**:
- Video not loading → Critical
- Like count delayed by few seconds → Acceptable

3. **Latency**

- Video start latency < 2 seconds
- API latency 50–100 ms
- Video should play with zero or negligible buffering
-

High Level Design (HLD):

The system follows a microservices-based architecture:

API Gateway

- Handles authentication, authorization, routing, and rate limiting

User Service

- User registration, login, JWT-based authentication
- Stores user data in MySQL

Subscription & Payment Service

- Manages subscription plans and payments
- Ensures strong consistency for transactions

Search Service

- Uses ElasticSearch for fast movie/TV show search

Video Metadata Service

- Stores metadata (title, description, thumbnails) in NoSQL DB

Video Upload & Processing Pipeline

- Uploaded videos are stored in Blob Storage
- Chunker Service splits videos into small segments
- Video Encoder generates multiple resolutions (480p, 720p, 1080p, 4K)
- Kafka coordinates asynchronous processing

Streaming Service

- Generates HLS (.m3u8) or DASH (.mpd) manifests
- Client fetches chunks based on bandwidth

CDN

- Caches most frequently watched video chunks
- Reduces latency and server load

Video Streaming Workflow:

- User clicks Play Video

- Client requests manifest file (.m3u8 / .mpd)
- Manifest contains URLs of video chunks with different bitrates
- Client measures network bandwidth
- Appropriate resolution chunks are fetched dynamically
- CDN serves cached chunks if available
- Video playback continues smoothly with adaptive bitrate

Scalability Solution:

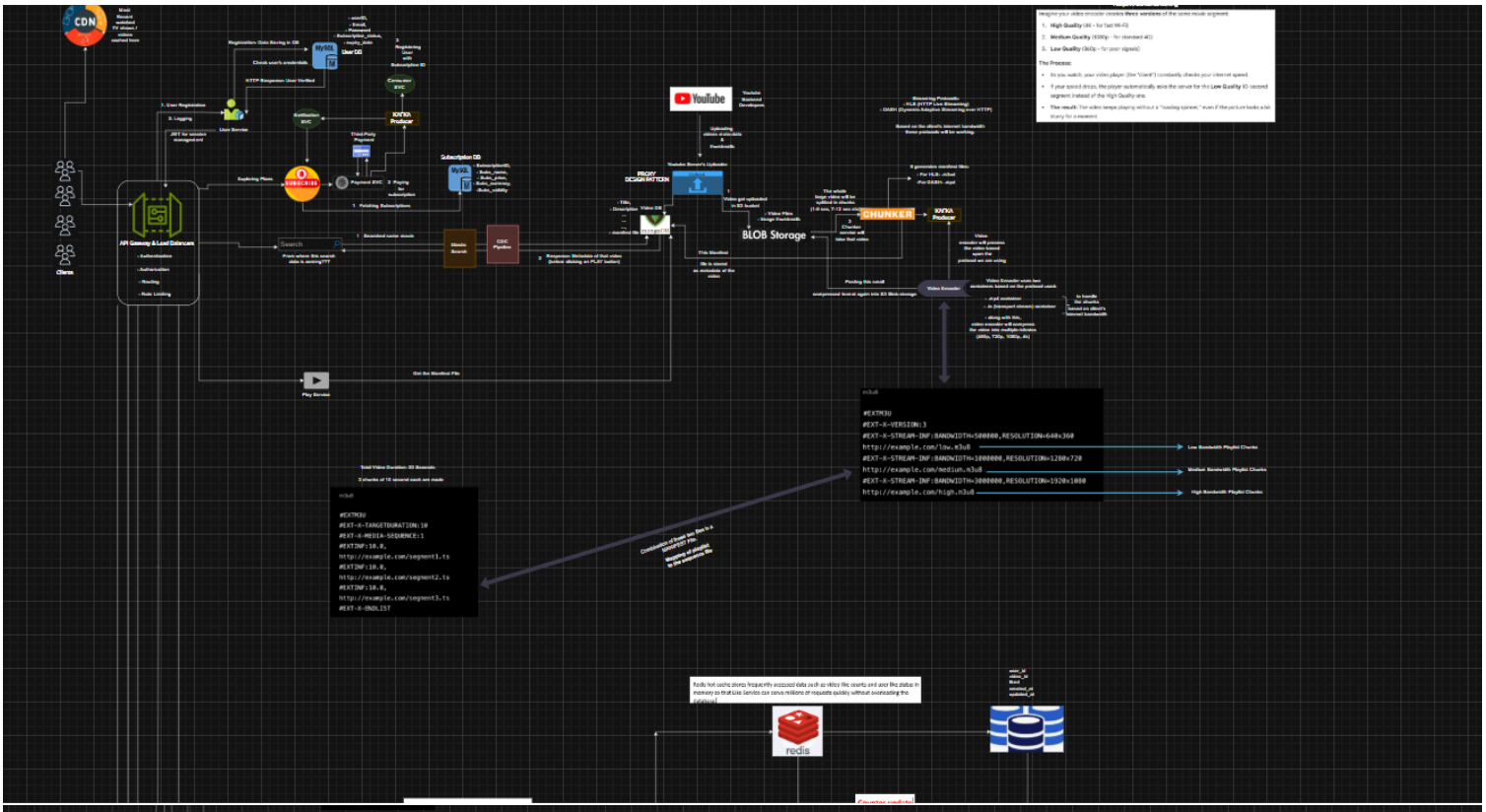
- Horizontal scaling of all microservices
- API Gateway acts as load balancer
- Kafka decouples video processing pipeline
- CDN caching minimizes latency and origin load
- Blob storage efficiently stores large video files
- Stateless services allow easy scaling

Like Service

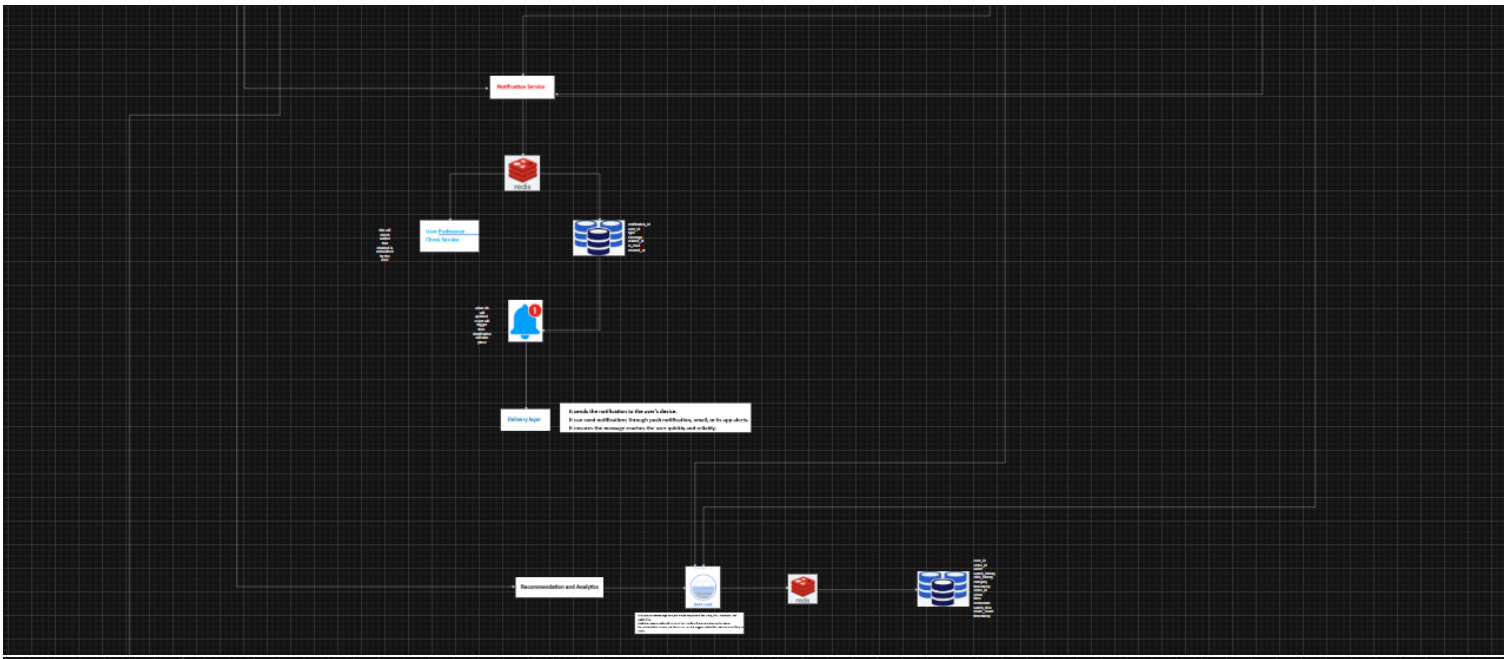
- User can like/unlike video
- Prevent duplicate likes (unique user_id + video_id)
- Redis stores real-time like counter
- Async update to Likes DB
- Flow:
- User → API Gateway → Like Service → Redis (INCR) Redis → Background Worker → Likes DB
- Reason for Cache:
  High write traffic must not overload database

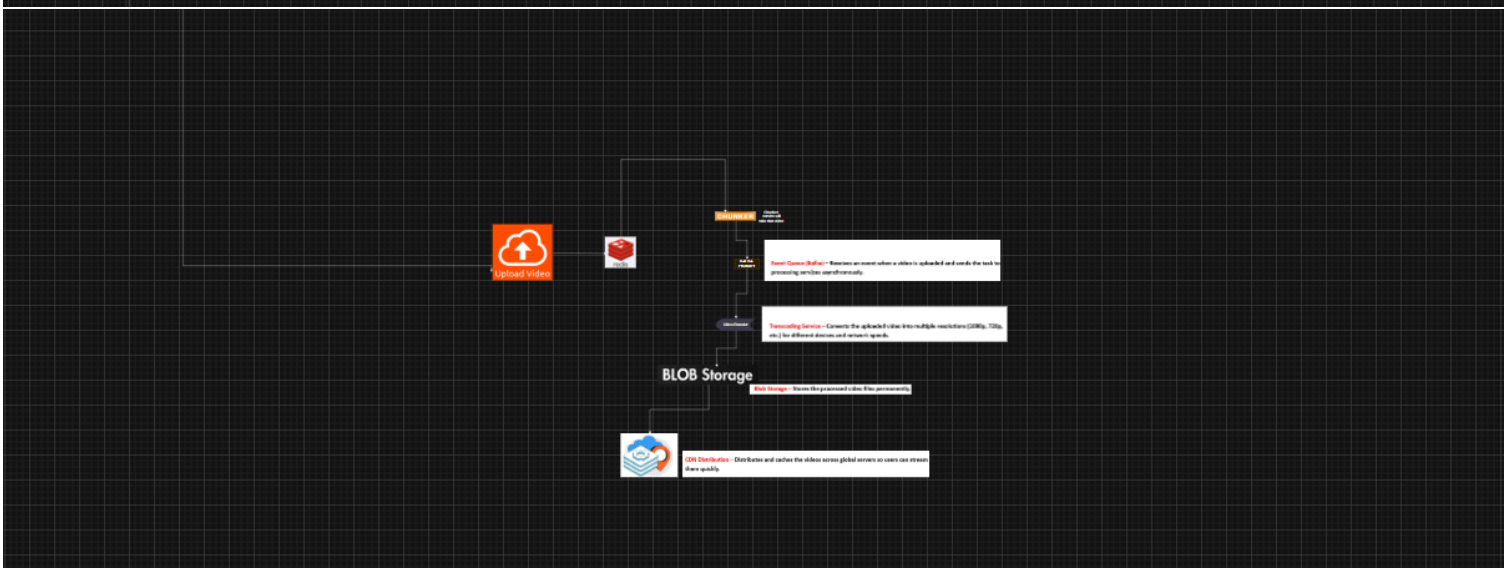
Comment Service

- Add comment
- Reply to comment
- Delete comment
- Pagination support
- Sort by latest/top


CDN
MySQL
User DB
Registration Data Saving in DB
Check user's credentials
HTTP Response: User Verified
Registering User with Subscription ID
Connector SVC
1. User Registration
2. Logging
User Service
JWT for session management
Notification SVC
KAFKA Producer
Third-Party Payment
Exploring Plans
Payment SVC
3. Paying for subscription
1. Fetching Subscriptions
Subscription DB
Subscriptions(PK, Subs_name, Subs_price, Subs_duration, Subs_validity)
API Gateway & Load Balancers
Authentication
Routing
Rate Limiting
Clients
Search
From where this search data is coming???
1. Searched some movie
Elastic Search
CDC Pipeline
2. Response: Metadata of that video (which showing on PLAY button)
YouTube
Youtube Backend Developers
Uploading videos as chunks & thumbnails
Youtube Server's Uploader
PROJECT DESCRIPTION
Title, Description, Video ID
Video got uploaded in S3 bucket
Video Files, Image thumbnails
BLOB Storage
Title Modified
File is stored as metadata of the video
Streaming Protocols: HLS (HTTP Live Streaming) - DASH (Dynamic Adaptive Streaming over HTTP)
Based on the client's internet bandwidth these protocols will be working.
If protocolos modified this: - For HLS: .m3u8 - For DASH: .mpd
This whole huge video will be splitted in chunks (1-10 sec, 5-12 sec size)
CHUNKER
Chunker service will take that video
KAFKA Producer
Video extender will process the chunks and store the protocol we are using
Passing this result
Video Encoder
Video Encoder uses has resolutions based on the protocol used.
.mpd resolutions
.m3u (Interrupted stream) resolutions
along with this, video encoder will compress the video into multiple bitrates (480p, 720p, 1080p, 4k)
In handle the chunks, based on client's internet bandwidth
compressed format again into S3 Blob storage
Play Service
Get the Modified File
m3u8
#EXTM3U
#EXT-X-VERSION:3
#EXT-X-STREAM-INF:BANDWIDTH=500000,RESOLUTION=640x360
http://example.com/low.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=1000000,RESOLUTION=1280x720
http://example.com/medium.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=3000000,RESOLUTION=1920x1080
http://example.com/high.m3u8
Low Bandwidth Playlist Chunks
Medium Bandwidth Playlist Chunks
High Bandwidth Playlist Chunks
Total Video Duration: 35 Seconds
3 chunks of 10 second each are made
m3u8
#EXTM3U
#EXT-X-TARGETDURATION:10
#EXT-X-MEDIA-SEQUENCE:1
#EXTINF:10.0,
http://example.com/segment1.ts
#EXTINF:10.0,
http://example.com/segment2.ts
#EXTINF:10.0,
http://example.com/segment3.ts
#EXT-X-ENDLIST
Combination of these two files is a MPEG-DASH file.
Number of chunks in the manifest file
Redis hot cache stores frequently accessed data such as video like counts and user like status in memory so that Like Service can serve millions of requests quickly without overloading the server.
redis
user_id
video_id
liked
created_at
updated_at

Redis hot cache stores frequently accessed data such as video like counts and user like status in memory so that Like Service can serve millions of requests quickly without overloading the server.
redis
user_id
video_id
liked
created_at
updated_at
LIKE SERVICE
Counter update
Kafka / PubSub Like event
Apache Cassandra is an open-source, distributed NoSQL database designed for handling massive amounts of data across many commodity servers, providing high scalability with no single point of failure. It uses a peer-to-peer, masterless architecture, and is known for its linear scalability and high write throughput.
Redis stores all user-level data: - Recently liked comments - Comment edit sessions - Rate limit counters - Temporary validation locks
redis
Comment Validator
cassandra
Comment Service
Comment Counter update
Kafka / PubSub comment event
Kafka / PubSub is used to send comment events to other services asynchronously.
1. When a comment is created, the Comment Service publishes a comment event to Kafka/PubSub.
2. Other services (Notification, Analytics, Recommendation) consume this event without blocking the comment request.
3. This allows the system to scale and process actions independently.


Notification Service
redis
User Preference Check Service
Delivery layer
It sends the notification to the user's device.
It can send notifications through push notification, email, or in-app alerts.
It ensures the message reaches the user quickly and reliably.
Recommendation and Analytics
redis


Upload Video
redis
CHUNKER
Event Queue (RabbitMQ) – Receives an event when a video is uploaded and sends the task to processing services asynchronously.
Transcoding Service – Converts the uploaded video into multiple resolutions (1080p, 720p, etc.) for different devices and network speeds.
BLOB Storage
Blob Storage – Stores the processed video files permanently.
CDN Distribution – Distributes and caches the videos across global servers so users can stream them quickly.

7. Learning Outcomes (What I Have Learnt):

- Understood how real-world OTT platforms like Netflix are designed
- Learned difference between functional and non-functional requirements
- Gained strong understanding of adaptive bitrate streaming (HLS/DASH)
- Understood CAP theorem trade-offs in streaming systems
- Learned how Kafka, CDN, and chunking pipelines work together
- Learned how to design highly available, low-latency distributed systems